Computer Vision Lab: Convolution and Filters

Introduction: The "Sliding Window" of Computer Vision

Today's lab is about the fundamental operation behind almost all modern image processing and Deep Learning: **Convolution**. We will move from building it from scratch to applying it for smoothing, sharpening, and optimizing for speed.

Assignment 3.1: Understanding Convolution Basics

Objective: Demystify the "black box" of convolution.

Concept for Students: Imagine looking at a large picture through a tiny square hole (the **kernel** or filter) carved out of a piece of paper. You slide this paper over the image. At every stop, you look at the pixels inside the hole, multiply them by the values written on your paper, sum them up, and write that number down on a new sheet. That is convolution.

Code Breakdown:

- **Padding:** Notice `pad_h` and `pad_w`. If we don't add a border of zeros (padding) around the image, our output image will shrink because the kernel can't center itself on the edge pixels.
- **The Loops:** The nested `for` loops (`range(h)`, `range(w)`) represent the "sliding" of the window.
- **The Math:** `output[i,j] = np.sum(region * kernel)` is the core. It is an **element-wise multiplication** followed by a **summation**.

Discussion Point:

- *Why use a library like OpenCV if we can write this?* Python loops are slow. OpenCV uses C++ and hardware optimization (SIMD) to do this millions of times faster.

Assignment 3.2: Image Smoothing (Linear Filters)

Objective: Remove noise by "blending" pixels.

Concept for Students: Noise usually manifests as sharp, random spikes in pixel intensity. To fix this, we assume that a pixel should look somewhat like its neighbors.

1. Mean Filter (The Box Filter):

- **How it works:** Takes the average of all pixels under the kernel.
- **The Kernel:** A 3×3 matrix where every value is $1/9$.
- **Pros/Cons:** It removes noise effectively but effectively "unfocuses" the image. Edges get blurred because the sharp transition is averaged out.

2. Gaussian Filter:

- **How it works:** It's a "weighted" average. Pixels closer to the center matter more; pixels further away matter less.
- **The Kernel:** Looks like a bell curve (highest in the middle, fading at edges).
- **Why it's better:** It preserves edges slightly better than the Mean filter because it respects the local structure more.

Metric - PSNR (Peak Signal-to-Noise Ratio):

- This measures quality. Higher PSNR = Better reconstruction (less noise).

Assignment 3.3: Sharpening Images

Objective: Enhance details by doing the opposite of smoothing.

Concept for Students: If blurring smoothes out changes, **sharpening** exaggerates them.

- **Edges** are areas where intensity changes rapidly (High Frequency).
- **Flat areas** are where intensity stays the same (Low Frequency).

The Laplacian Filter: The Laplacian calculates the *second derivative* of the image intensity.

- If the image is flat, the sum is 0 (gray).

- If there is an edge, it produces a high positive value on one side and a high negative value on the other.

Unsharp Masking (The Formula):

$$\text{Sharpened Image} = \text{Original Image} + (\text{Scaling Factor} \times \text{Edges})$$

We literally "add" the edges back onto the original image to make them pop.

Assignment 3.4: Separable Filters (Optimization)

Objective: Make convolution faster.

Concept for Students: This is a pure computer science optimization.

- **2D Convolution:** For a 5×5 kernel, you do **25 multiplications** for *every single pixel* in the image.
- **Separable Filter:** A 2D Gaussian kernel can be broken into two 1D kernels (a vertical vector and a horizontal vector).
  1. Convolve the image with the vertical column (5×1).
  2. Convolve that result with the horizontal row (1×5).

The Efficiency Gain:

- **2D:** K^2 operations per pixel. (For $K = 5$, that's 25 ops).
- **Separable:** $K + K$ operations per pixel. (For $K = 5$, that's 10 ops).
- **Result:** The separable filter is **2.5x faster** in this case! As the kernel gets bigger (e.g., 11×11), the speed difference becomes massive (121 ops vs 22 ops).

Assignment 3.5: Non-Linear vs Linear Filtering

Objective: Why math (averaging) fails on "Salt and Pepper" noise.

The Problem: Salt and Pepper noise sets random pixels to **0 (Black)** or **255 (White)**.

- **Linear Filter (Mean):** If you have a block of pixels like `[10, 10, 10, 255, 10]`, the average becomes `59`. The white dot (`255`) drags the average up, so the bright "salt" noise just becomes a gray smudge. It doesn't disappear; it spreads.

The Solution: Median Filter (Non-Linear):

- **How it works:** It collects all pixels under the kernel, **sorts** them, and picks the middle value.
- **Example:** `[10, 10, 10, 255, 10]` $\rightarrow$ Sorted: `[10, 10, 10, 10, 255]` $\rightarrow$ Median: `10`.
- **Result:** The `255` is mathematically ignored as an outlier. The noise is completely removed, and the edge is preserved perfectly.

Conclusion:

- **Convolution** is great for general smoothing and feature extraction.
- **Sorting (Median)** is essential for outlier removal.

Topic 1: The Mechanics of Convolution (Assignment 3.1)

Concept: The "Sliding Window" Start by explaining that an image is just a matrix of numbers. Convolution is the process of modifying a pixel's value based on its neighbors.

- **The Analogy:** Imagine holding a small magnifying glass (the **Kernel**) over a large map (the **Image**). You look at only the area under the glass, perform a calculation, write down the result, and then slide the glass one step to the right.
- **The Kernel:** This is the most important concept. The kernel is a small matrix (usually 3×3 or 5×5) that acts as a "filter."
  - *If the kernel values are all equal?* It blurs the image.
  - *If the kernel has positive center and negative edges?* It sharpens or detects edges.
  - *Key Takeaway:* The **operation** (convolution) is always the same; the **kernel values** determine the effect.
- **The Math (Weighted Sum):** For every position, we do element-wise multiplication between the image patch and the kernel, then sum them up. This is often called "Multiply-Accumulate."
- **Padding:** Ask the students: *"What happens to the pixels at the very edge of the image?"*
  - Without padding, the kernel falls off the edge, and we can't calculate a value.
  - Padding adds a border (usually zeros) so the output image stays the same size as the input.

Topic 2: Smoothing & Linear Filters (Assignment 3.2)

Concept: Removing Noise by Averaging Noise usually appears as sharp, sudden spikes in pixel intensity (high frequency). Smoothing filters try to suppress these spikes by blending neighboring pixels.

- **Mean Filter (The "Democracy" Approach):**
  - **Theory:** Every pixel in the 5×5 neighborhood gets an equal vote. We take the simple average.
  - **Result:** It effectively removes noise, but it has a major drawback: it treats *edges* (which we want to keep) the same as *noise* (which we want to lose). This makes the image look "unfocused."
- **Gaussian Filter (The "Weighted" Approach):**
  - **Theory:** Pixels closer to the center are more important than pixels far away. The kernel values follow a bell curve (Gaussian

distribution).

    - **Result:** The center pixel contributes most to the final value. This results in a smoother image that preserves the "structure" better than a mean filter because it doesn't smear distant pixels into the current one as aggressively.

- **Key Discussion:** Why is this called "Linear"? Because the output is just a weighted sum of inputs ($a \cdot x + b \cdot y + \ldots$).

Topic 3: Sharpening & Laplacian (Assignment 3.3)

Concept: Enhancing Differences Sharpening is the opposite of smoothing. If smoothing removes high frequencies (changes), sharpening boosts them.

- **The Laplacian (2nd Derivative):**
    - Explain that in calculus, the **derivative** measures change.
    - A **Zero derivative** means a flat area (constant color).
    - A **High derivative** means an edge (rapid color change).
    - The Laplacian kernel (e.g., center 4, neighbors -1) calculates this change. When you convolve with it, flat areas become black (0), and edges light up.
- **Unsharp Masking:**
    - This name is counter-intuitive. It comes from darkroom photography.
    - **The Formula:** $\text{Sharpened} = \text{Original} + (\text{Original} - \text{Blurred})$.
    - Basically, we isolate the details (edges) and *add* them back onto the original image to make them pop.

Topic 4: Computational Efficiency (Assignment 3.4)

Concept: Working Smarter, Not Harder This is a critical Computer Science concept applied to vision.

- **The Problem:** A 2D convolution is expensive.
    - For a 5×5 kernel, you perform **25 multiplications** for *every single pixel* in the image.
- **The Solution (Separability):**

    - Some kernels (like Gaussian) are "separable." This means the 2D matrix can be created by multiplying a vertical vector (5×1) and a horizontal vector (1×5).
    - Instead of doing one big 2D convolution, we do two small 1D convolutions.
- **The Math:**
    - **2D:** Cost is N^2 (25 ops).
    - **Separable:** Cost is $N + N$ (10 ops).
    - **Impact:** For a kernel size of 5, separable is **2.5x faster**. For a kernel size of 11, it is **5.5x faster** (121 ops vs 22 ops). This is how real-time apps (like Instagram filters) work fast.

Topic 5: Non-Linear Filtering (Assignment 3.5)

Concept: Why Math fails on "Salt & Pepper" Sometimes, averaging is the wrong tool.

- **The Scenario:** Salt & Pepper noise randomly turns pixels to pure White (255) or pure Black (0). These are outliers.
- **Why Mean Filter Fails:**
    - Imagine a list of pixel values: `[10, 11, 10, 255, 10]`. (The `255` is the noise).
    - If you take the **Average**, you get `59`.
    - The `255` drags the average up. The bright white dot doesn't disappear; it just becomes a gray smudge.
- **Why Median Filter Wins:**
    - The Median filter **Sorts** the values: `[10, 10, 10, 10, 255]`.
    - It picks the middle value: `10`.
    - The `255` is completely ignored because it is an outlier.
- **Conclusion:**
    - **Convolution (Linear)** creates new pixel values (averages).
    - **Median (Non-Linear)** just *selects* an existing value from the neighborhood. It is not a convolution because you cannot define a fixed kernel matrix to perform a "sort and select" operation.

We will take a **Single Input Image (Matrix)** and apply **Three Different Kernels** to see how the output changes based *only* on the numbers in the kernel.

The Setup

1. The Input Image (5x5 Matrix)

Imagine a simple grayscale image.

- **0** = Black
- **10** = Bright Gray
- This image has a clear **Vertical Edge** (a jump from 0 to 10).

$$\text{Input Image} = \begin{bmatrix} 0 & 0 & 10 & 10 & 0 \\ 0 & 0 & 10 & 10 & 0 \\ 0 & 0 & 10 & 10 & 0 \\ 0 & 0 & 10 & 10 & 0 \\ 0 & 0 & 10 & 10 & 0 \end{bmatrix}$$

Experiment 1: Vertical Edge Detection Kernel

Goal: Find where the image changes from Dark to Light.

The Kernel (3x3):

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

- **Logic:** It subtracts the left pixels from the right pixels.
  - If Left and Right are same ($0 - 0$ or $10 - 10$), result is **0** (No Edge).
  - If Left is small (0) and Right is big (10), result is **Positive** (Edge detected).

The Math (Center Pixel at Row 2, Col 2): Let's place the kernel over the center of the image (highlighted in bold):

$$\text{Region} = \begin{bmatrix} 0 & \mathbf{0} & 10 \\ 0 & \mathbf{0} & 10 \\ 0 & \mathbf{0} & 10 \end{bmatrix}$$

Operation: Multiply the Region by the Kernel (element-wise) and sum:

$$\begin{gathered} (-1 \times 0) + (0 \times 0) + (1 \times 10) = 10 \\ (-1 \times 0) + (0 \times 0) + (1 \times 10) = 10 \\ (-1 \times 0) + (0 \times 0) + (1 \times 10) = 10 \end{gathered}$$

Sum: $10 + 10 + 10 = \mathbf{30}$

The Output Matrix (Central part): The output shows high numbers (30) exactly where the vertical line is.

$$\text{Output} = \begin{bmatrix} \dots & \dots & \dots & \dots & \dots \\ \dots & 0 & \mathbf{30} & 0 & \dots \\ \dots & 0 & \mathbf{30} & 0 & \dots \\ \dots & 0 & \mathbf{30} & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix}$$

Result: We found the edge!

Experiment 2: Box Blur (Mean Filter)

Goal: Smooth out the image by averaging.

The Kernel (3x3):

$$K = \frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The Math (Same Center Pixel): Again, overlay on the transition area:

$$\text{Region} = \begin{bmatrix} 0 & 0 & 10 \\ 0 & 0 & 10 \\ 0 & 0 & 10 \end{bmatrix}$$

Operation: Sum all pixels and divide by 9. Sum = $0 + 0 + 10 + 0 + 0 + 10 + 0 + 0 + 10 = 30$ Average = $30/9 \approx \mathbf{3.33}$

The Output: Instead of a sharp jump from 0 to 10, we get a transition value (3.33).

$$\text{Output} \approx \begin{bmatrix} \dots & \dots & \dots & \dots & \dots \\ \dots & 0 & \mathbf{3.3} & \mathbf{6.6} & \dots \\ \dots & 0 & \mathbf{3.3} & \mathbf{6.6} & \dots \\ \dots & 0 & \mathbf{3.3} & \mathbf{6.6} & \dots \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix}$$

Result: The sharp "0 to 10" edge is now a smooth slope "0 $\rightarrow$ 3.3 $\rightarrow$ 6.6 $\rightarrow$ 10". The image is blurred.

Experiment 3: Identity Kernel (The "Do Nothing" Filter)

Goal: Prove that a specific kernel preserves the original image.

The Kernel:

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Logic:** Ignore all neighbors (0s), just keep the center pixel (1).

The Math:

$$\text{Region} = \begin{bmatrix} 0 & 0 & 10 \\ 0 & 0 & 10 \\ 0 & 0 & 10 \end{bmatrix}$$

Only the center matters: $(0 \times 0) + (0 \times 1) + (10 \times 0) = \mathbf{0}$

Wait, let's move the window one step right to where the 10 is:

$$\text{Region} = \begin{bmatrix} 0 & 10 & 10 \\ 0 & 10 & 10 \\ 0 & 10 & 10 \end{bmatrix}$$

Center calculation: $(10 \times 1) =$

Result: The Output Matrix is exactly the same as the Input Matrix.